

UNIVERSITÉ DE KINSHASA

Master 1 – Intelligence Artificielle

Implémentation de l'algorithme des nuées dynamiques

From scratch en Python

Étudiant :	KABONGO TSHIATA ETIENNE
Niveau :	M1 IA
Cours :	Science des données
Sujet :	Implémentation de l'algorithme des nuées dynamiques (from scratch)
Année académique :	2025–2026

Kinshasa, septembre 2026

Table des matières

1	Introduction	2
2	Problématique et objectifs	2
3	Fondements théoriques	2
3.1	Partition et centre de gravité	2
3.2	Distance et fonction objectif	3
3.3	Alternance des deux phases	3
4	Algorithme proposé	3
4.1	Pseudo-code	3
4.2	Critères d'arrêt et complexité	4
4.3	Choix de conception	4
5	Implémentation from scratch	4
6	Protocole expérimental	4
6.1	Jeu de données	4
6.2	Paramètres	5
7	Résultats et analyse	5
7.1	Partition obtenue	5
7.2	Convergence	6
7.3	Choix du nombre de classes	6
7.4	Validation comparative	7
8	Limites et pistes d'amélioration	7
9	Reproductibilité et dépôt GitHub	7
10	Conclusion	8

1 Introduction

L'apprentissage non supervisé cherche à extraire une structure à partir de données qui ne possèdent pas d'étiquette connue. Parmi ses tâches centrales, la classification automatique (*clustering*) consiste à constituer des groupes homogènes : les observations d'un même groupe doivent être proches, tandis que celles de groupes différents doivent être éloignées.

L'algorithme des nuées dynamiques, étroitement lié à la méthode des k -moyennes, alterne une phase d'affectation aux centres les plus proches et une phase de représentation de chaque classe par son centre de gravité. Cette famille de méthodes trouve notamment ses racines dans les travaux de MacQueen [4] et de Lloyd [3].

Ce travail présente les fondements mathématiques, une implémentation personnelle en Python et une expérimentation reproductible. L'objectif n'est pas d'appeler une fonction prête à l'emploi : le calcul des distances, les affectations, les centres, l'inertie et les critères d'arrêt sont programmés directement.

2 Problématique et objectifs

Soit une matrice de données $X \in \mathbb{R}^{n \times p}$. Comment partitionner ses n observations en k classes compactes sans disposer d'étiquettes ? L'enjeu est de construire une procédure compréhensible, correcte et robuste aux aléas de l'initialisation.

Les objectifs spécifiques sont :

- formaliser le problème d'optimisation et les étapes de l'algorithme ;
- coder toutes les opérations fondamentales sans KMeans externe ;
- gérer les classes vides et les initialisations défavorables ;
- observer la convergence et apprécier le choix de k ;
- vérifier séparément la cohérence des résultats avec une référence.

3 Fondements théoriques

3.1 Partition et centre de gravité

Une partition $P = (C_1, \dots, C_k)$ divise l'ensemble des observations en classes disjointes. Pour une classe non vide C_j , son centre de gravité est la moyenne de ses observations :

$$\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i. \quad (1)$$

Ce centre minimise la somme des distances euclidiennes au carré aux membres de la classe [2].

3.2 Distance et fonction objectif

La distance euclidienne entre une observation $\mathbf{x}_i$ et un centre $\boldsymbol{\mu}_j$ est

$$d(\mathbf{x}_i, \boldsymbol{\mu}_j) = \sqrt{\sum_{r=1}^p (x_{ir} - \mu_{jr})^2}. \quad (2)$$

Comme la racine carrée est croissante, l'affectation peut employer directement la distance au carré. L'inertie intra-classe à minimiser vaut

$$J(P, \boldsymbol{\mu}) = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2. \quad (3)$$

3.3 Alternance des deux phases

À centres fixés, chaque observation rejoint la classe de son centre le plus proche :

$$z_i = \arg \min_{1 \leq j \leq k} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2. \quad (4)$$

À partition fixée, les centres sont recalculés par

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{|C_j^{(t)}|} \sum_{\mathbf{x}_i \in C_j^{(t)}} \mathbf{x}_i. \quad (5)$$

Chacune de ces phases ne peut augmenter l'objectif (3). Comme il existe un nombre fini de partitions, la procédure atteint un point fixe en un nombre fini d'étapes. Ce point est toutefois un optimum local, pas nécessairement l'optimum global.

4 Algorithme proposé

4.1 Pseudo-code

1. choisir k observations distinctes comme centres initiaux ;
2. calculer toutes les distances au carré ;
3. affecter chaque observation au centre le plus proche ;
4. remplacer chaque centre par la moyenne de sa classe ;
5. si une classe est vide, replacer son centre sur une observation mal représentée ;
6. calculer l'inertie et mémoriser sa valeur ;
7. arrêter si le déplacement global des centres est inférieur à ε , si la partition est stable, ou si la limite d'itérations est atteinte ; sinon retourner à l'étape 2 ;
8. répéter l'ensemble avec plusieurs initialisations et conserver la solution de plus faible inertie.

4.2 Critères d'arrêt et complexité

Le déplacement est mesuré par la norme de Frobenius :

$$\Delta^{(t)} = \|M^{(t+1)} - M^{(t)}\|_F. \quad (6)$$

L'arrêt intervient lorsque $\Delta^{(t)} \leq \varepsilon$ ou lorsque deux partitions successives sont identiques. Pour T itérations, le coût dominant du calcul des distances est $O(Tnkp)$ et le stockage de la matrice des distances demande $O(nk)$.

4.3 Choix de conception

L'implémentation utilise une graine pseudo-aléatoire pour assurer la reproductibilité. Plusieurs démarrages réduisent le risque d'un mauvais optimum local. En cas de classe vide, le nouveau centre est choisi parmi les points qui contribuent le plus à l'inertie courante. Une méthode `predict` permet enfin d'affecter de nouvelles observations aux centres appris.

5 Implémentation from scratch

Le cœur du calcul des distances est le suivant :

```
1 def _distances_carre(X, centres):
2     differences = X[:, np.newaxis, :] - centres[np.newaxis, :, :]
3     return np.sum(differences * differences, axis=2)
```

L'affectation et l'inertie ne nécessitent ensuite que l'indice du minimum de chaque ligne et la somme des distances retenues :

```
1 distances = self._distances_carre(X, centres)
2 etiquettes = np.argmin(distances, axis=1)
3 inertie = np.sum(distances[np.arange(X.shape[0]), etiquettes])
```

NumPy sert ici uniquement à représenter les tableaux et à effectuer les opérations arithmétiques. La logique de clustering est entièrement contenue dans la classe `NueesDynamiques`. Le fichier complet est fourni dans `src/nuees_dynamiques.py`.

6 Protocole expérimental

6.1 Jeu de données

Le jeu synthétique contient 450 observations bidimensionnelles, réparties en trois nuages gaussiens de 150 points. Leurs moyennes génératrices sont $(-3, -2)$, $(0, 2, 3, 2)$ et $(3, 6, -0, 8)$, avec des dispersions différentes. Une graine fixée à 42 permet de régénérer exactement les mêmes données. Le CSV produit est inclus dans le projet.

6.2 Paramètres

L'expérience principale emploie $k = 3$, au maximum 200 itérations, une tolérance de 10^{-6} et 20 initialisations. L'étude du coude compare ensuite les valeurs $k = 1, \dots, 8$ avec 15 initialisations pour chacune.

7 Résultats et analyse

7.1 Partition obtenue

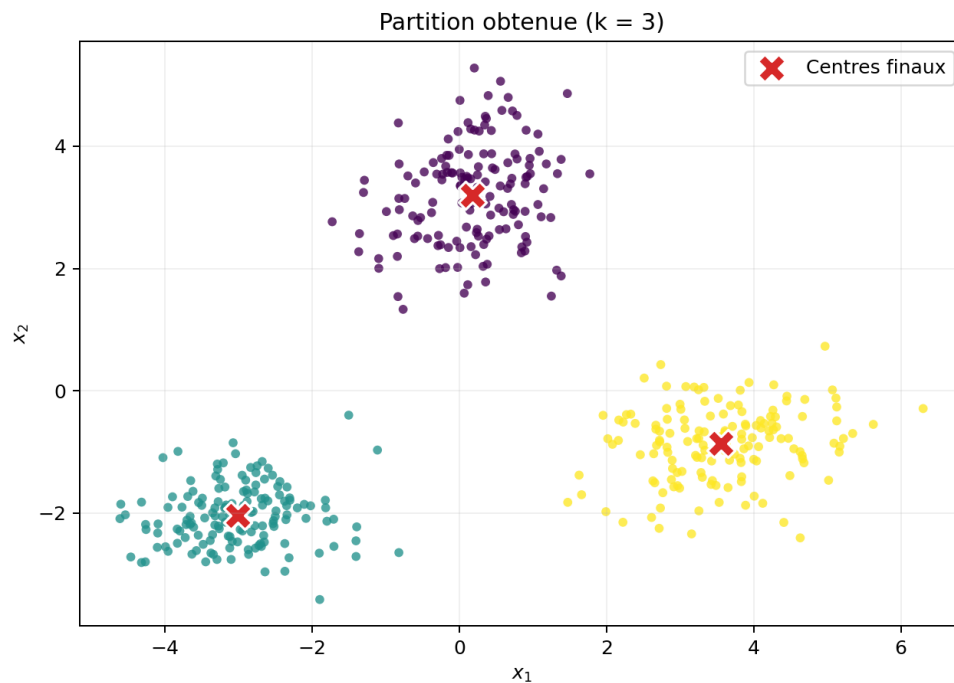


Figure 1: Classes découvertes et centres de gravité finaux.

Les trois groupes de la figure 1 sont nettement séparés. Les centres calculés sont proches des moyennes génératrices :

$$(0,183; 3,203), \quad (-3,014; -2,035), \quad (3,554; -0,862).$$

L'ordre des classes n'a pas de signification : une permutation des indices décrit la même partition.

7.2 Convergence

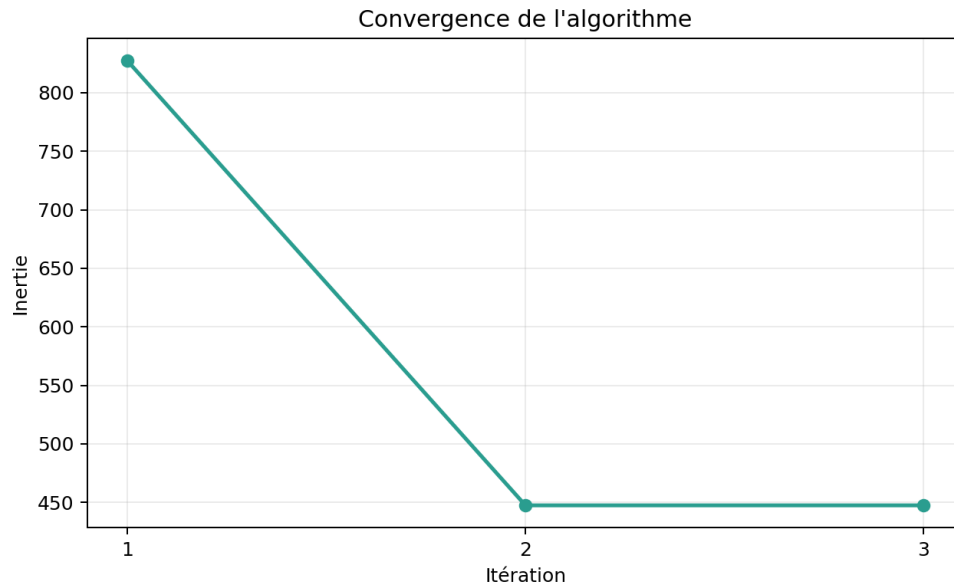


Figure 2: Évolution de l'inertie de la meilleure exécution.

La meilleure exécution atteint une inertie de 447,520831 en deux itérations. La décroissance puis la stabilisation observées sont conformes au principe d'optimisation alternée.

7.3 Choix du nombre de classes

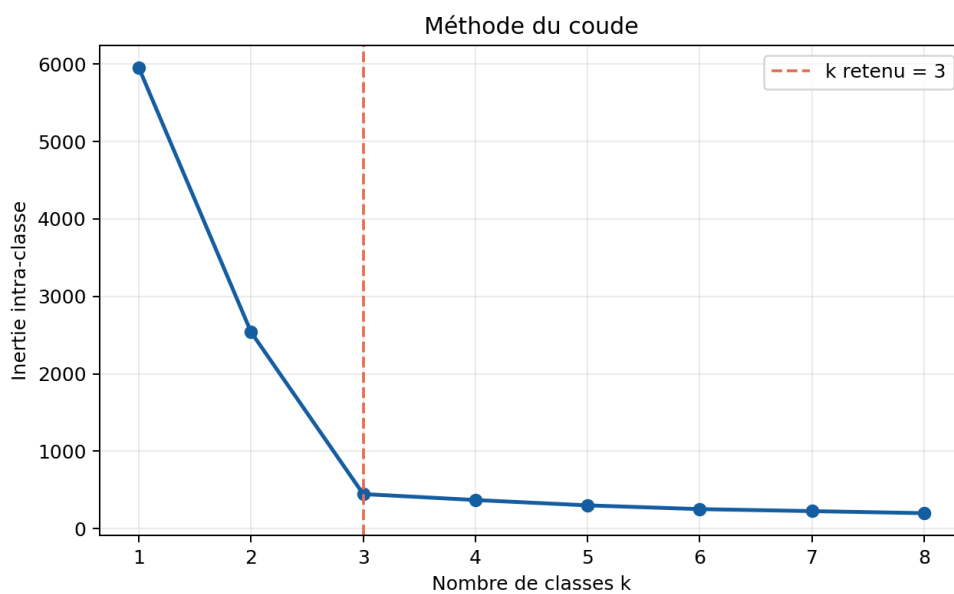


Figure 3: Inertie en fonction du nombre de classes.

Le passage de $k = 2$ (2538,877) à $k = 3$ (447,521) apporte un gain majeur. Après $k = 3$, les diminutions deviennent beaucoup plus progressives. Le coude confirme donc le choix de trois classes pour ce jeu de données.

7.4 Validation comparative

La comparaison n'intervient qu'après l'exécution du code personnel. Avec les mêmes données, l'implémentation de référence décrite dans la documentation de scikit-learn [5] obtient elle aussi une inertie de 447,520831. L'indice de Rand ajusté (ARI) entre les deux partitions vaut 1,000, de même que l'ARI entre notre partition et les classes génératrices. Cette égalité montre, pour l'expérience considérée, que les partitions sont identiques à une permutation des étiquettes près.

Table 1: Synthèse quantitative de l'expérience.

Indicateur	Valeur
Nombre d'observations	450
Nombre de variables	2
Nombre de classes	3
Itérations de la meilleure exécution	2
Inertie finale	447,520831
ARI avec scikit-learn	1,000000
ARI avec les classes génératrices	1,000000

8 Limites et pistes d'amélioration

L'algorithme exige que k soit fixé à l'avance et reste sensible à l'initialisation, malgré les redémarrages. La distance euclidienne le rend plus adapté aux classes compactes et approximativement sphériques ; il décrit mal les formes non convexes. Les variables de grande échelle peuvent dominer les distances, d'où la nécessité de standardiser les données réelles. Enfin, les valeurs aberrantes déplacent les moyennes.

Des extensions possibles comprennent l'initialisation k -means++ [1], la standardisation automatique, l'indice de silhouette pour guider le choix de k , le traitement par mini-lots pour de grands volumes et la comparaison avec DBSCAN ou un clustering hiérarchique.

9 Reproductibilité et dépôt GitHub

Le projet contient le code, les tests, les données, les figures et les sources LaTeX. Après téléchargement, l'expérience s'exécute par :

```
pip install -r requirements.txt
python experiments/experimentation.py
pytest -q
```

Le dépôt peut être publié à l'adresse suivante, à compléter après sa création :

<https://github.com/VOTRE-COMPTE/nuees-dynamiques-from-scratch>

Le lien ne peut pas être rendu définitif avant la création effective du dépôt sur le compte GitHub de l'étudiant.

10 Conclusion

Ce travail a construit l'algorithme des nuées dynamiques depuis ses opérations élémentaires. La classe développée calcule manuellement les distances, alterne affectation et mise à jour des centres, mesure l'inertie, traite les classes vides, conserve l'historique de convergence et prédit la classe de nouveaux points. Sur un jeu reproductible de 450 observations, elle retrouve parfaitement les trois groupes générateurs et concorde avec la référence externe. Cette réalisation met ainsi en évidence à la fois la simplicité, l'efficacité et les limites de la méthode.

Références

- [1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- [2] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.
- [3] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- [4] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967.
- [5] Scikit-learn developers. *KMeans Documentation*, 2026. Consulté le 19 septembre 2026.